

Task Distribution

Person 1 — Control Unit @orion

Deliverables:

- Full control unit module
- Truth table for all opcodes

Outputs:

- Lui, PcSrc, MemRead, MemWrite, AluOp/AluCtrl, MemToReg, AluSrc1, AluSrc2, RegWrite, Branch, Jump

Why this is a full job:

- This is the “brain” of the CPU.
- Requires reading the instruction formats and mapping each opcode to the correct control signals.
- Needs careful testing with sample instructions.

Integration responsibility:

Provide a clean document listing each opcode → control signals so the rest of the team can wire confidently.

Person 2 — ALU Control + Branch/Jump Unit @eren

These two naturally pair because they both depend on funct3/funct7 and ALU results.

Deliverables:

- ALU Control Unit
 - Maps (AluOp, funct3, funct7) → 4-bit ALU code
 - Includes BLTU and BGEU from the spec
- Branch & Jump Unit
 - Computes next PC
 - Handles:
 - PC + 4
 - PC + offset
 - JAL
 - JALR (rs1 + offset)
 - Branch conditions (BEQ/BNE/BLT/BGE/BLTU/BGEU)

Why this is a full job:

- ALU control is mechanical but must be exact.
- Branch/jump logic is subtle and easy to get wrong.
- This person becomes the “control-flow correctness” owner.

Person 3 (Kai) — Memories + Immediate Generator + Muxes

This person handles all the “data movement” blocks.

Deliverables:

- Instruction Memory module
- Data Memory module
- Immediate Generator
- Parameterized 2:1 Mux module (used everywhere)

Why this is a full job:

- Memory modules must match skeleton I/O exactly.
- Immediate generator is critical for branch offsets, loads, stores, JAL, JALR, etc.
- Muxes must be parameterized and reusable.

Integration responsibility:

Provide a clean list of signal widths so the top-module person can wire without guessing.

Person 4 — Top Module (Integration Engineer) @Dawn

This is the most important role for avoiding chaos.

Deliverables:

- Top-level module that instantiates:
 - PC register
 - Instruction memory
 - Decoder
 - Register file
 - Immediate generator
 - ALU
 - ALU control
 - Control unit
 - Data memory
 - Branch/jump unit

- All muxes

Why this is a full job:

- No logic, but *all wiring*.
- Requires understanding the datapath diagram in the PDF and matching skeleton ports exactly.
- This person ensures the whole CPU actually runs.

Integration responsibility:

- Maintain a shared signal map (names, widths, owners).
- Run the first full simulation.

Why this split works

- Each person owns a **module boundary**, not random lines.
- No two people touch the same file.
- The top-module person becomes the “glue,” not a coder.
- Everyone can test their module independently before integration.

Optional: If someone is weaker at Verilog

Give them **muxes + instruction memory**, and shift immediate generator to the ALU-control person. But the 4-way split above is the cleanest.

Control Unit

Control branch logic. I tried to understand how each instruction moves through the datapath.

Instruction Type	Opcode	Branch	Jump	ALUOp	Lui	MemRead	MemToReg	MemWrite	PCSrc	ALUSrc1	ALUSrc2	RegWrite
R-type (add, sub, etc.)	0110011	0	0	010	0	0	0	0	0	0	0	1
I-type (addi, andi, ori, etc.)	0010011	0	0	011	0	0	0	0	0	0	1	1
Load (lw, lh, lb, etc.)	0000011	0	0	000	0	1	1	0	0	0	1	1
Store (sw, sh, sb)	0100011	0	0	000	0	0	0	1	0	0	1	0
Branch (beq, bne, etc.)	1100011	1	0	001	0	0	0	0	1	0	0	0
U-type(lui)	0110111	0	0	000	1	0	0	0	0	0	1	1
U-type(auipc)	0010111	0	0	000	0	0	0	0	0	1	1	1
J-type	1101111	0	1	000	0	0	0	0	1	1	1	1
JALR	1100111	0	1	000	0	0	0	0	1	0	1	1

For the ALU Control Unit:

My module will pass the ALUOp data into the ALU control unit. It is a 3-bit bus, and it decides what type of operation we need from the ALU for the given opcode.

My implementation assumes the following:

ALUOp	Meaning	Used for
000	ADD (default)	loads, stores, JAL, JALR, AUIPC
001	Branch compare	BEQ/BNE/BLT/BGE/etc.
010	R-type	ADD/SUB/AND/OR/XOR/etc.
011	I-type ALU	ADDI/ANDI/ORI/etc.

So if ALUOp = 000: simply add rs1 and rs2.

If ALUOp = 001: do a comparison and output a 1(true) or 0(false).

ALUOp = 010: use the funct3+funct7 input data to determine the ALU operation.

ALUOp = 011: use funct3/7 to determine ALU operation.